PERSONAL FINANCE

EXPENSE TRACKER



PREPARED BY

Himanshu Kumar

TRACK • REVIEW • PLAN

SNO	CONTENTS	PAGE NO.
1	INTRODUCTION	1
2	KEY FEATURES	2-4
3	TECHNOLOGY STACK	5-7
4	MODULES IMPORTED	8
5	USER DEFINED FUNCTION	9-12
6	SOURCE CODE	13-30
7	OUTPUT	31-39
8	MY SQL SCHEMA	41
9	BIBLIOGRAPHY	42

INTRODUCTION

In today's fast-paced digital world, effective financial management has become an essential part of everyday life. With the increase in personal expenses and digital transactions, individuals often find it difficult to track their spending and maintain control over their budget.

This Computer Science project focuses on the development of an **Expense Tracker Application** that helps users record, manage, and analyze their daily expenses in an organized manner. The application allows users to create accounts, set budgets, record expenses under different categories, and view summaries of their spending patterns.

The project integrates **Python programming, Tkinter for graphical user interface, and MySQL for database management** to ensure secure storage and efficient retrieval of user data. Analytical features such as expense summaries, budget comparisons, and graphical representations help users understand their financial behavior.

Through this project, we aim to highlight the importance of responsible financial planning and demonstrate how technology can assist individuals in managing their expenses effectively.

KEY FEATURES

1. User-Friendly Interface

The Expense Tracker application provides a simple, interactive, and visually appealing graphical user interface developed using **Python Tkinter**. The interface is designed to be easy to understand and navigate, allowing users to access different features such as login, expense entry, analytics, and summaries without any technical difficulty.

2. Secure Login and Sign-Up System

The application includes a secure **user authentication system** that allows users to create accounts and log in using a username and password. User credentials, budget details, and account information are stored safely in a **MySQL database**, ensuring data persistence and personalized access.

3. Expense Management Module

Users can record their daily expenses by entering details such as **date, category, description, amount, and payment method**. The application stores all expense records in the database, enabling users to insert and search expense data efficiently.

4. Budget Tracking and Overview

The system allows users to set a monthly budget during registration. The **Budget Overview** feature compares the total expenses with the allocated budget and displays the summary in a tabular format. This helps users understand whether they are under budget, on budget, or overspending.

5. Expense Analytics and Graphical Representation

The application provides graphical analysis of expenses using **Matplotlib**. Users can view:

- Category-wise expense distribution using pie charts
- Comparison between total expenses and budget using bar graphs
  These visual representations make it easier to analyze spending patterns.

6. Expense Summary and Insights

The **Summaries module** displays key financial insights such as:

- Total expenses
- Budget amount
- Most spent category
- Category-wise total expenses
  Visual indicators are also used to show whether the user is spending wisely or exceeding the budget.

7. Account Management

Users can update their account details such as **password and budget**. The application also provides an option to delete user accounts along with all related expense records, ensuring complete data management.

8. Suggestion System

Based on the comparison between expenses and budget, the system generates simple financial suggestions such as:

- Overspending warning
- Budget limit reached
- Positive feedback for controlled spending
  This feature helps users improve their financial habits.

TECHNOLOGY STACK

The **Expense Tracker System** is developed as a desktop-based application using Python. The project integrates a graphical user interface, a relational database, and backend logic to provide an efficient and user-friendly experience. Below is the technology stack used in the development:

1. Programming Language

•Python

Python is the core programming language used in this project. It is simple, powerful, and well-suited for building desktop applications, handling database operations, and performing data analysis. Python enables rapid development and easy maintenance of the system.

2. Graphical User Interface (GUI)

•Tkinter

Tkinter is Python's standard GUI library used to design and implement the user interface of the Expense Tracker. It provides interactive components such as buttons, labels, entry fields, frames, and tables, allowing users to manage expenses, view summaries, and navigate the application easily.

3. Database

•MySQL

MySQL is used as the relational database management system for storing user credentials, budget details, and expense records. It

ensures secure, structured, and efficient storage of data, enabling quick retrieval and updates.

4. Database Connectivity

• MySQL Connector for Python

The mysql-connector-python library is used to establish a connection between the Python application and the MySQL database. It allows seamless execution of SQL queries such as insert, update, delete, and select operations directly from the application.

5. Data Visualization

•Matplotlib

Matplotlib is used to generate graphical representations such as pie charts and bar graphs. These visualizations help users analyze their spending patterns and compare total expenses with their budget effectively.

6. Platform

•Desktop(Windows)

The Expense Tracker application is designed to run on desktop systems, particularly Windows. It does not require internet connectivity, making it suitable for personal use, educational projects, and offline financial tracking.

MODULES IMPORTED

Module	description
tkinter	Standard Python library used for creating graphical user interfaces (GUIs). It provides components such as windows, buttons, labels, frames, and entry fields used throughout the Expense Tracker application.
tkinter.ttk	Submodule of Tkinter that provides themed and modern widgets like Treeview, which is used to display tabular data such as expense records and summaries.
mysql.connector	Python module used to connect the application with the MySQL database. It enables execution of SQL queries for inserting, updating, deleting, and retrieving expense and user data.
matplotlib.pyplot	Library used for data visualization. It is used to generate pie charts and bar graphs to analyze expenses and compare total expenses with the user's budget.
tkinter.filedialog	Submodule of Tkinter used to open file dialog windows. It can be used for future enhancements like importing or exporting expense data.

USER DEFINED FUNCTIONS

➢ AUTHENTICATION MODULE

Function Name	Purpose
logsig()	Displays the initial screen with Login and Sign-up options.
login()	Authenticates user credentials using MySQL database and allows access to the application.
sign_up()	Collects new user details and stores them in the signup table.

➢ DASHBOARD MODULE

Function Name	Purpose
MAIN1(username)	Displays the main dashboard with navigation menu after successful login.
dashboard(username)	Opens the dashboard screen with options like budget overview and graphs.
budgetoverview(username)	Displays the total expense and budget details of the user.
graph(username)	Generates a pie chart showing category-wise expense distribution.

➢ EXPENSE MANAGEMENT MODULE

Function Name	Purpose
E_Management(username)	Provides options to insert or search expense records.
insert(username)	Displays a form to insert new expense details into the database.
search(username)	Fetches and displays all expense records of the logged-in user in a table.

➢ ANALYTICS MODULE

Function Name	Purpose
analytics(username)	Compares total expenses with the budget and displays a bar graph.

➢ ACCOUNT MANAGEMENT MODULE

Function Name	Purpose
A_management(username)	Provides options to update or delete user account information.
update(username)	Updates the user's password and budget details.
delete(username)	Deletes the user account and related expense records from the database.

➢ REPORT & SUMMARY MODULE

Function Name	Purpose
Report(username)	Opens the report section of the application.
suggestion(username)	Provides spending suggestions based on expense and budget comparison.
Summaries(username)	Displays total expense, budget, most spent category, and summary table.

MAIN PROGRAM

Function Name	Purpose
logsig()	Displays the starting screen with Login and Sign-up options.
login()	Verifies user credentials and grants access to the system.
sign_up()	Registers a new user into the database.
MAIN1(username)	Displays the main menu and navigation buttons after login.

MYSQL SCHEMA

MySQL 8.4 Command Line Client

```
mysql> describe signup;
+-----------+--------------+------+-----+---------+-------+
| Field     | Type         | Null | Key | Default | Extra |
+-----------+--------------+------+-----+---------+-------+
| User_Name | varchar(100) | NO   | PRI | NULL    |       |
| Password  | varchar(100) | YES  |     | NULL    |       |
| Budget    | int          | YES  |     | NULL    |       |
| Month     | varchar(20)  | YES  |     | NULL    |       |
| Year      | int          | YES  |     | NULL    |       |
+-----------+--------------+------+-----+---------+-------+
5 rows in set (1.72 sec)

mysql>
```

MySQL 8.4 Command Line Client

```
mysql> describe expenses;
+-------------+--------------+------+-----+---------+-------+
| Field       | Type         | Null | Key | Default | Extra |
+-------------+--------------+------+-----+---------+-------+
| Uid         | varchar(100) | YES  |     | NULL    |       |
| Dot         | date         | YES  |     | NULL    |       |
| Category    | varchar(100) | YES  |     | NULL    |       |
| Description | text         | YES  |     | NULL    |       |
| Amount      | int          | YES  |     | NULL    |       |
| Payment     | varchar(20)  | YES  |     | NULL    |       |
+-------------+--------------+------+-----+---------+-------+
6 rows in set (0.04 sec)
```

SOURCE CODE

```
from tkinter import *
import tkinter as tk
from tkinter import ttk
import mysql.connector
conn = mysql.connector.connect(user="root",
                               host="localhost",
                               database="exptracker",
                               password="anshu2113")
curr = conn.cursor()
w = Tk()
w.geometry("1300x610")
w.img12_veltro = PhotoImage(file="veltro1.png")
def logsig():
    w.img12_global = PhotoImage(file="global.png")
    f1 = Frame(w, width=1300, height=610)
    f1.place(x=0, y=0)
    L1 = Label(f1, image=w.img12_global)
    L1.place(x=0, y=0)
    b1 = Button(f1, text="Sign_up", width=16, relief="flat", bg="#BA1415", fg="white",
font=("rockwell",12), command=sign_up)
    b1.place(x=478, y=333)
    b2 = Button(f1, text="Login", width=16, relief="flat", bg="#BA1415", fg="white",
font=("rockwell",12), command=login)
    b2.place(x=709, y=333)
f2 = Frame(w, width=1300, height=610)
f2.place(x=0, y=0)
L2 = Label(f2, image=w.img12_veltro)
L2.place(x=0, y=0)
b1 = Button(f2, text="Get Started", width=13, relief="flat", bg="#BA2628", fg="white", font=("Open
Sans",15), command=logsig)
b1.place(x=151, y=323)

def login():
    global imgbg, f3
    f3 = Frame(w, width=1300, height=610)
    f3.place(x=0, y=0)
    imgbg = PhotoImage(file="login.png")
    L3 = Label(f3, image=imgbg)
    L3.place(x=0, y=0, relwidth=1, relheight=1)
    t2 = Entry(f3, width=20, relief="flat", bg="#08050D",fg="white", font=("rockwell", 14))
    t2.place(x=430, y=261)
    t3 = Entry(f3, width=20, relief="flat", bg="#08050D",fg="white", font=("rockwell", 14))
    t3.place(x=430, y=340)
    def connecting():
        username = t2.get()
```

```
        password = t3.get()
        query = "select * from signup where User_Name='{}' and password='{}'".format(username,
password)
        curr.execute(query)
        r = curr.fetchone()
        if r:
            print("Login Successful!")
            MAIN1(username)
        f3.place_forget()
    b = Button(f3, text="Login", width=18, relief="flat", fg="white", bg="#CA0610", font=("rockwell",
13), command=connecting)
    b.place(x=567, y=408)

def sign_up():

    global img1
    f4 = Frame(w, width=1308, height=610, bg="light blue")
    f4.place(x=0, y=0)
    img1 = PhotoImage(file="signup.png")
    L4 = Label(f4, image=img1)
    L4.place(x=0, y=0, relwidth=1, relheight=1)

    Label(f4, text="UserName", bg="#0D0A0F", fg="white", font=("rockwell",11,"bold"),
width=22).place(x=365,y=211)
    Label(f4, text="Password", bg="#0D0A0F", fg="white", font=("rockwell",11,"bold"),
width=22).place(x=365,y=248)
    Label(f4, text="Budget", bg="#0D0A0F", fg="white", font=("rockwell",11,"bold"),
width=22).place(x=365,y=288)
    Label(f4, text="Month", bg="#0D0A0F", fg="white", font=("rockwell",11,"bold"),
width=22).place(x=365,y=325)
    Label(f4, text="Year", bg="#0D0A0F", fg="white", font=("rockwell",11,"bold"),
width=22).place(x=365,y=363)

    t2 = Entry(f4, width=20, bg="#0D0A0F", fg="white", relief="flat", font=("rockwell",14))
    t2.place(x=759,y=211)
    t3 = Entry(f4, width=20, bg="#0D0A0F", fg="white", relief="flat", font=("rockwell",14))
    t3.place(x=759,y=248)
    t4 = Entry(f4, width=20, bg="#0D0A0F", fg="white", relief="flat", font=("rockwell",14))
    t4.place(x=759,y=288)
    t5 = Entry(f4, width=20, bg="#0D0A0F", fg="white", relief="flat", font=("rockwell",14))
    t5.place(x=759,y=325)
    t6 = Entry(f4, width=20, bg="#0D0A0F", fg="white", relief="flat", font=("rockwell",14))
    t6.place(x=759,y=363)
    def getInfo():
        username = t2.get()
        password = t3.get()
        budget = t4.get()
        month = t5.get()
        year = t6.get()
```

```
        query = "insert into signup values('{}','{}',{},'{}',{})".format(username, password, budget,
month, year)
        curr.execute(query)
        conn.commit()
        f4.place_forget()
    Button(f4, text="Submit", width=16, relief="flat", bg="#C90D15", font=("rockwell",14),
command=getInfo).place(x=564,y=445)

def MAIN1(username):
    global imgbg
    f5 = Frame(w, width=1300, height=610, bg="light blue")
    f5.place(x=0, y=0)
    imgbg = PhotoImage(file="blackred12.png")
    L5 = Label(f5, image=imgbg)
    L5.place(x=0, y=0)
    f6 = Frame(f5, width=200, height=610, bg="#1E1E21")
    f6.place(x=0, y=0)

    b1 = Button(f5, text="Dashboard", width=15, font=("rockwell", 12), command=lambda:
dashboard(username))
    b1.place(x=20, y=20)

    b2 = Button(f5, text="Expense Management", width=15, font=("rockwell", 12),
command=lambda: E_Management(username))
    b2.place(x=20, y=60)

    b3 = Button(f5, text="Analytics", width=15, font=("rockwell", 12), command=lambda:
analytics(username))
    b3.place(x=20, y=100)

    b4 = Button(f5, text="Account Management", width=15, font=("rockwell", 12),
command=lambda: A_management(username))
    b4.place(x=20, y=140)

    b5 = Button(f5, text="Suggestion/Report", width=15, font=("rockwell", 12), command=lambda:
Report(username))
    b5.place(x=20, y=180)

    b6 = Button(f5, text="Summaries", width=15, font=("rockwell", 12), command=lambda:
Summaries(username))
    b6.place(x=20, y=220)



def dashboard(username):
    global f7, img1

    f7 = Frame(w, width=1300, height=610)
```

```
    f7.place(x=200, y=0)
    img1 = PhotoImage(file="blackred.png")
    L4 = Label(f7, image=img1)
    L4.place(x=0, y=0)


    b1 = Button(f7, text="budgetoverview", width=15, font=("rockwell", 12), command=lambda:
budgetoverview(username))
    b1.place(x=20, y=20)
    b2 = Button(f7, text="graph", width=15, font=("rockwell", 12), command=lambda:
graph(username))
    b2.place(x=20, y=60)



def graph(username):


    global img1
    f8 = Frame(f7, width=1308, height=610)
    f8.place(x=0, y=0)
    img1 = PhotoImage(file="blackred.png")
    L4 = Label(f7, image=img1)
    L4.place(x=0, y=0)

    L6 = Label(f8, width=20, text="User", bg="black", fg="white", font=("rockwell", 12, "bold"))
    L6.place(x=10, y=40)
    t = Entry(f8, width=20, font=("rockwell", 12))
    t.insert(0, username)
    t.config(state='disabled')
    t.place(x=250, y=40)
    query = "SELECT category, SUM(amount) AS total_amount FROM expenses WHERE Uid='{}'
GROUP BY category".format(username)
    curr.execute(query)
    L1 = []
    L2 = []
    r = curr.fetchall()
    for i in r:
        L1.append(i[0])
        L2.append(i[1])
    import matplotlib.pyplot as plt
    plt.pie(L2, labels=L1, autopct='%1.1f%%', startangle=140)
    plt.title('Total Expenses by Category')
    plt.axis('equal')
    plt.tight_layout()
    plt.show()
def budgetoverview(username):
    global f7, img1
    f9 = Frame(f7, width=1300, height=615)
```

```
    f9.place(x=0, y=0)
    f9.pack_propagate(False)
    img1 = PhotoImage(file="blackred.png")
    L4 = Label(f9, image=img1)
    L4.place(x=0, y=0)
    L7 = Label(f9, width=20, text="Username", bg="black", fg="white", font=("rockwell", 12, "bold"))
    L7.place(x=10, y=40)
    t1 = Entry(f9, width=20, font=("rockwell", 12))
    t1.insert(0, username)
    t1.config(state='disabled')
    t1.place(x=250, y=40)
    def sugg():
        curr.execute("SELECT SUM(amount) FROM expenses WHERE Uid = %s", (username,))
        expense_result = curr.fetchone()

        total_expense = expense_result[0] if expense_result and expense_result[0] else 0
        curr.execute("SELECT budget FROM signup WHERE User_Name = %s", (username,))

        budget_result = curr.fetchone()

        budget = budget_result[0] if budget_result and budget_result[0] else 0

        columns = ("Uid", "Total Expense", "Budget")

        tree = ttk.Treeview(f9, columns=columns, show='headings', height=1)

        for col in columns:
            tree.heading(col, text=col)
            tree.column(col, width=110)

        tree.insert("", "end", values=(username, total_expense, budget))
        tree.place(x=100, y=130)
    Button(f9, text="Get Summary", width=18, font=("rockwell", 12), command=sugg).place(x=250,
y=80)

def E_Management(username):
    global f10, img1
    f10 = Frame(w, width=1300, height=610)
    f10.place(x=200, y=0)
    img1 = PhotoImage(file="blackred.png")
    L4 = Label(f10, image=img1)
    L4.place(x=0, y=0)
    b1 = Button(f10, text="Insert", width=15, font=("rockwell", 12), command=lambda:
insert(username))
    b1.place(x=20, y=20)
    b2 = Button(f10, text="Search", width=15, font=("rockwell", 12), command=lambda:
search(username))
    b2.place(x=20, y=60)
```

```
def insert(username):
    global f10, img1
    f11 = Frame(w, width=1300, height=615, bg="lavender")
    f11.place(x=200, y=0)
    img1 = PhotoImage(file="blackred.png")
    L4 = Label(f11, image=img1)
    L4.place(x=0, y=0)

    Label(f11, width=20, text="*U_id*", bg="black", fg="white", font=("Helvetica", 12,
"bold")).place(x=10, y=40)

    Label(f11, width=20, text="Date", bg="Black", fg="white", font=("rockwell", 12,
"bold")).place(x=10, y=90)

    Label(f11, width=20, text="Category", bg="Black", fg="white", font=("rockwell", 12,
"bold")).place(x=10, y=130)

    Label(f11, width=20, text="Description", bg="Black", fg="white", font=("rockwell", 12,
"bold")).place(x=10, y=170)

    Label(f11, width=20, text="Amount", bg="Black", fg="white", font=("rockwell", 12,
"bold")).place(x=10, y=210)

    Label(f11, width=20, text="Payment", bg="Black", fg="white", font=("rockwell", 12,
"bold")).place(x=10, y=250)

    t1 = Entry(f11, width=20, font=("rockwell", 12))
    t1.insert(0, username)
    t1.config(state='disabled')
    t1.place(x=250, y=40)

    t2 = Entry(f11, width=20, font=("rockwell", 12)); t2.place(x=250, y=90)

    t3 = Entry(f11, width=20, font=("rockwell", 12)); t3.place(x=250, y=130)

    t4 = Entry(f11, width=20, font=("rockwell", 12)); t4.place(x=250, y=170)

    t5 = Entry(f11, width=20, font=("rockwell", 12)); t5.place(x=250, y=210)

    t6 = Entry(f11, width=20, font=("rockwell", 12)); t6.place(x=250, y=250)

    def getInfo2():
        Uid = t1.get()
        date = t2.get()
        category = t3.get()
        description = t4.get()
        amount = t5.get()
        payment = t6.get()
```

```
        query = "INSERT INTO expenses VALUES ('{}', '{}', '{}', '{}', {}, '{}')".format(Uid, date, category,
description, amount, payment)
        curr.execute(query)
        conn.commit()
    Button(f11, text="Submit", width=20, font=("rockwell", 12), command=getInfo2).place(x=205,
y=330)

def search(username):
    global f10, img1
    f12 = Frame(f10, width=1300, height=615, bg="Lavender")
    f12.place(x=0, y=0)
    f12.pack_propagate(False)
    img1 = PhotoImage(file="blackred.png")
    L4 = Label(f12, image=img1)
    L4.place(x=0, y=0)
    Label(f12, width=20, text="*User_id*", bg="black", fg="white", font=("rockwell", 12,
"bold")).place(x=10, y=40)
    t1 = Entry(f12, width=20, font=("rockwell", 12))
    t1.insert(0, username)
    t1.config(state='disabled')
    t1.place(x=250, y=40)
    def connecting2():
        Uid = t1.get()
        query = "SELECT * FROM expenses WHERE Uid='{}'".format(Uid)
        curr.execute(query)
        r = curr.fetchall()
        if not r:
            print("No expenses found.")
        else:
            columns = ("Uid", "date", "category", "description", "amount", "payment")
            tree = ttk.Treeview(f12, columns=columns, show='headings')
            for col in columns:
                tree.heading(col, text=col)
                tree.column(col, width=100)
            for i in r:
                tree.insert("", END, values=i)
            tree.place(x=20, y=100)
    Button(f12, text="SEARCH", width=12, font=("rockwell", 8),
command=connecting2).place(x=380, y=40)

def analytics(username):
    global f13, img1
    f13 = Frame(w, width=1300, height=615)
    f13.place(x=200, y=0)
    f13.pack_propagate(False)
    img1 = PhotoImage(file="blackred.png")
    L4 = Label(f13, image=img1)
    L4.place(x=0, y=0)
```

```
    Label(f13, width=20, text="User", bg="black", fg="white", font=("rockwell", 12,
"bold")).place(x=10, y=40)
    t = Entry(f13, width=20, font=("rockwell", 12))
    t.insert(0, username)
    t.config(state='disabled')
    t.place(x=250, y=40)
    curr.execute("SELECT SUM(amount) FROM expenses WHERE Uid = %s", (username,))
    te = curr.fetchone()[0] or 0
    curr.execute("SELECT budget FROM signup WHERE User_Name = %s", (username,))
    budget = curr.fetchone()[0] or 0
    import matplotlib.pyplot as plt
    categories = ['Total Expense', 'Budget']
    values = [te, budget]
    colors = ['lavender', 'lightcoral']
    plt.bar(categories, values, color=colors)
    plt.title(f'Expense vs Budget for {username}')
    plt.ylabel('Amount')
    plt.tight_layout()
    plt.show()


def A_management(username):
    global f14, img1
    f14 = Frame(w, width=1300, height=610, bg="lavender")
    f14.place(x=200, y=0)
    img1 = PhotoImage(file="blackred.png")
    L4 = Label(f14, image=img1)
    L4.place(x=0, y=0)
    b1 = Button(f14, text="Update", width=15, font=("rockwell", 12), command=lambda:
update(username))
    b1.place(x=20, y=20)
    b2 = Button(f14, text="Delete", width=15, font=("rockwell", 12), command=lambda:
delete(username))
    b2.place(x=20, y=60)


def update(username):
    global f14,img1


    f15 = Frame(w, width=1300, height=610, bg="light blue")
    f15.place(x=200, y=0)
    img1 = PhotoImage(file="blackred.png")
    L4 = Label(f15, image=img1)
    L4.place(x=0, y=0)

    Label(f15, width=20, text="User", bg="black", fg="white", font=("rockwell", 12,
"bold")).place(x=10, y=40)
    t = Entry(f15, width=20, font=("rockwell", 12))
```

```
    t.insert(0, username)
    t.config(state='disabled')
    t.place(x=250, y=40)
    Label(f15, width=20, text="Password", bg="black", fg="white", font=("rockwell", 12,
"bold")).place(x=10, y=80)
    t2 = Entry(f15, width=11, font=("rockwell", 10))
    t2.place(x=250, y=80)
    Label(f15, width=20, text="Budget", bg="black", fg="white", font=("rockwell", 12,
"bold")).place(x=10, y=120)
    t3 = Entry(f15, width=11, font=("rockwell", 10))
    t3.place(x=250, y=120)
    def getInfo3():
        User_Name = t.get()
        password = t2.get()
        budget = t3.get()
        curr.execute( "UPDATE signup SET password='{}', budget={} WHERE
User_Name='{}'".format(password, budget, User_Name))
        conn.commit()
        Label(f15, width=30, text="RECORD UPDATED SUCCESSFULLY", bg="black", fg="white",
font=("rockwell", 12, "bold")).place(x=100, y=200)
    Button(f15, text="Submit", width=20, font=("rockwell", 12), command=getInfo3).place(x=200,
y=300)



def delete(username):
    global f14, img1
    f16 = Frame(f14, width=1300, height=615, bg="Lavender")
    f16.place(x=0, y=0)
    f16.pack_propagate(False)
    img1 = PhotoImage(file="blackred.png")
    L4 = Label(f16, image=img1)
    L4.place(x=0, y=0)
    Label(f16, width=20, text="Username", bg="black", fg="white", font=("rockwell", 12,
"bold")).place(x=10, y=40)
    t1 = Entry(f16, width=20, font=("rockwell", 12))
    t1.insert(0, username)
    t1.config(state='disabled')
    t1.place(x=250, y=40)
    def getInfo4():
        uid = t1.get()
        query = "DELETE FROM signup WHERE User_Name='{}'".format(uid)
        query2 = "DELETE FROM expenses WHERE Uid='{}'".format(uid)
        curr.execute(query)
        curr.execute(query2)
        conn.commit()
        Label(f16, width=30, text="RECORD DELETED SUCCESSFULLY", bg="black", fg="white",
font=("rockwell", 12, "bold")).place(x=100, y=100)
```

```
    Button(f16, text="Submit", width=20, font=("rockwell", 12), command=getInfo4).place(x=200,
y=300)


def Report(username):
    global f17, img1
    f17 = Frame(w, width=1300, height=610)
    f17.place(x=200, y=0)
    img1 = PhotoImage(file="blackred.png")
    L4 = Label(f17, image=img1)
    L4.place(x=0, y=0)
    b2 = Button(f17, text="Suggestion", width=15, font=("rockwell", 12), command=lambda:
suggestion(username))
    b2.place(x=20, y=60)

def suggestion(username):
    global f17, img1
    f18 = Frame(f17, width=1300, height=615, bg="Lavender")
    f18.place(x=0, y=0)
    f18.pack_propagate(False)
    img1 = PhotoImage(file="blackred.png")
    L4 = Label(f18, image=img1)
    L4.place(x=0, y=0)
    L6 = Label(f18, width=20, text="Username", bg="black", fg="white", font=("rockwell", 12,
"bold"))
    L6.place(x=10, y=40)
    t1 = Entry(f18, width=20, font=("rockwell", 12))
    t1.insert(0, username)
    t1.config(state='disabled')
    t1.place(x=250, y=40)
    def sugg():
        curr.execute("SELECT SUM(amount) FROM expenses WHERE Uid = %s", (username,))
        eresult = curr.fetchone()
        if eresult is None or eresult[0] is None:
            te = 0
        else:
            te = eresult[0]
        curr.execute("SELECT budget FROM signup WHERE User_Name = %s", (username,))
        bresult = curr.fetchone()
        if bresult is None or bresult[0] is None:
            budget = 0
        else:
            budget = bresult[0]
        columns = ("Uid", "Total Expense", "Budget")
        tree = ttk.Treeview(f18, columns=columns, show='headings', height=1)
        for col in columns:
            tree.heading(col, text=col)
            tree.column(col, width=110)
        tree.insert("", "end", values=(username, te, budget))
```

```
        tree.place(x=100, y=130)
        if te > budget:
            msg = "You are spending more than your budget"
            "Reduce unnecessary expenses and track your spending more carefully to regain control."
        elif te == budget:
            msg = "You have reached your budget limit."
            "Avoid additional spending and monitor expenses closely until the next cycle."
        else:
            msg = "You are managing your budget well."
            " Continue tracking expenses and consider saving the remaining amount."
        Label(f18, text=msg, bg="black", fg="white", font=("rockwell", 12, "bold"),
width=40).place(x=100, y=250)
    Button(f18, text="Get Suggestion", width=18, font=("rockwell", 12),
command=sugg).place(x=250, y=80)




def Summaries(username):
    global f19
    f19 = Frame(w, width=1100, height=610, bg="lavender")
    f19.place(x=200, y=0)

    f19.bg_img = PhotoImage(file="blackred.png")
    bg = Label(f19, image=f19.bg_img)
    bg.place(x=0, y=0, relwidth=1, relheight=1)

    Label(f19, text=f"Username: {username}",
          font=("rockwell", 12), bg="lavender").place(x=10, y=10)

    curr.execute("SELECT SUM(amount) FROM expenses WHERE Uid=%s", (username,))
    totale = curr.fetchone()[0] or 0

    Label(f19, text=f"Total Expense: ¹{totale}",
          font=("rockwell", 12), bg="lavender").place(x=10, y=40)

    curr.execute("SELECT budget FROM signup WHERE User_Name=%s", (username,))
    budget = curr.fetchone()[0] or 0

    Label(f19, text=f"Budget: ¹{budget}",
          font=("rockwell", 12), bg="lavender").place(x=10, y=70)

    curr.execute("""
        SELECT category, SUM(amount)
        FROM expenses
        WHERE Uid=%s
        GROUP BY category
        ORDER BY SUM(amount) DESC
        LIMIT 1
```

```
    """, (username,))

    r = curr.fetchone()
    cat = r[0] if r else "N/A"
    amt = r[1] if r else 0

    Label(f19,text=f"Most Spent Category: {cat} (¹{amt})",font=("rockwell",
12),bg="lavender").place(x=10, y=400)

    columns = ("Category", "Date", "Total Amount")
    tree = ttk.Treeview(f19, columns=columns, show="headings", height=8)

    for col in columns:
        tree.heading(col, text=col)
        tree.column(col, width=150)

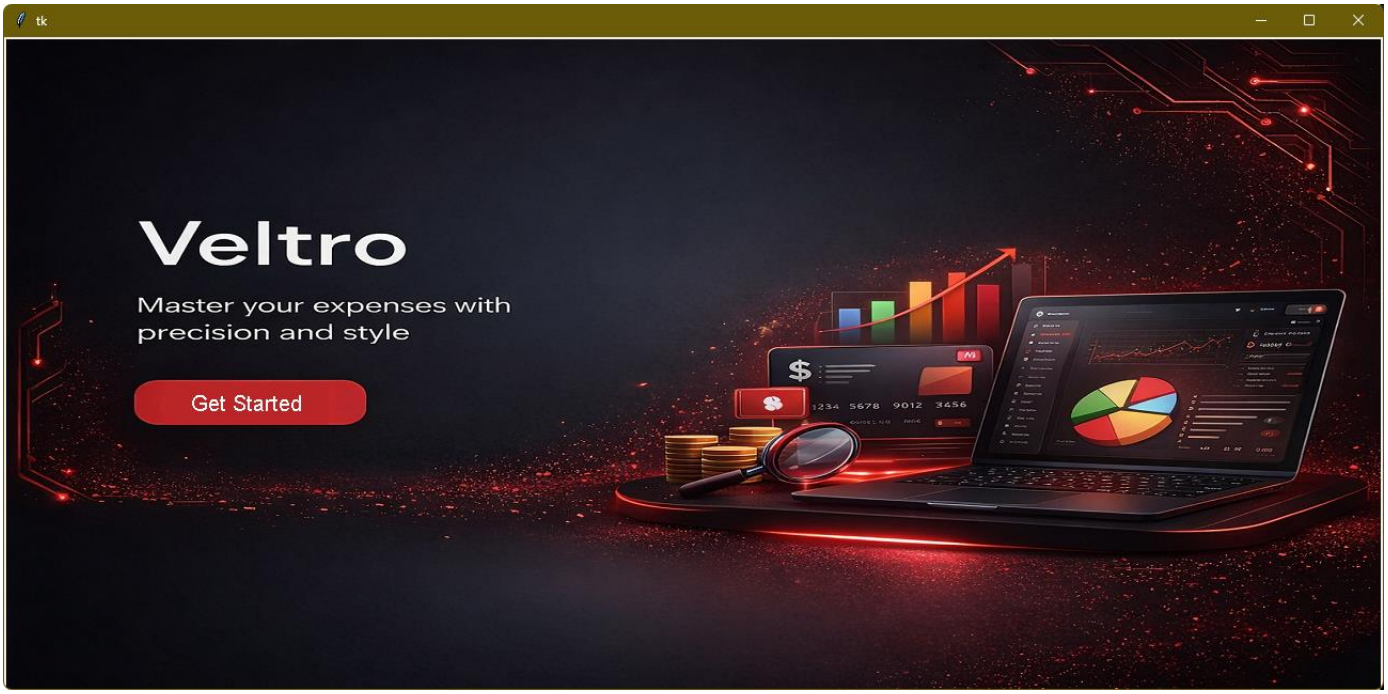
    tree.place(x=10, y=110)

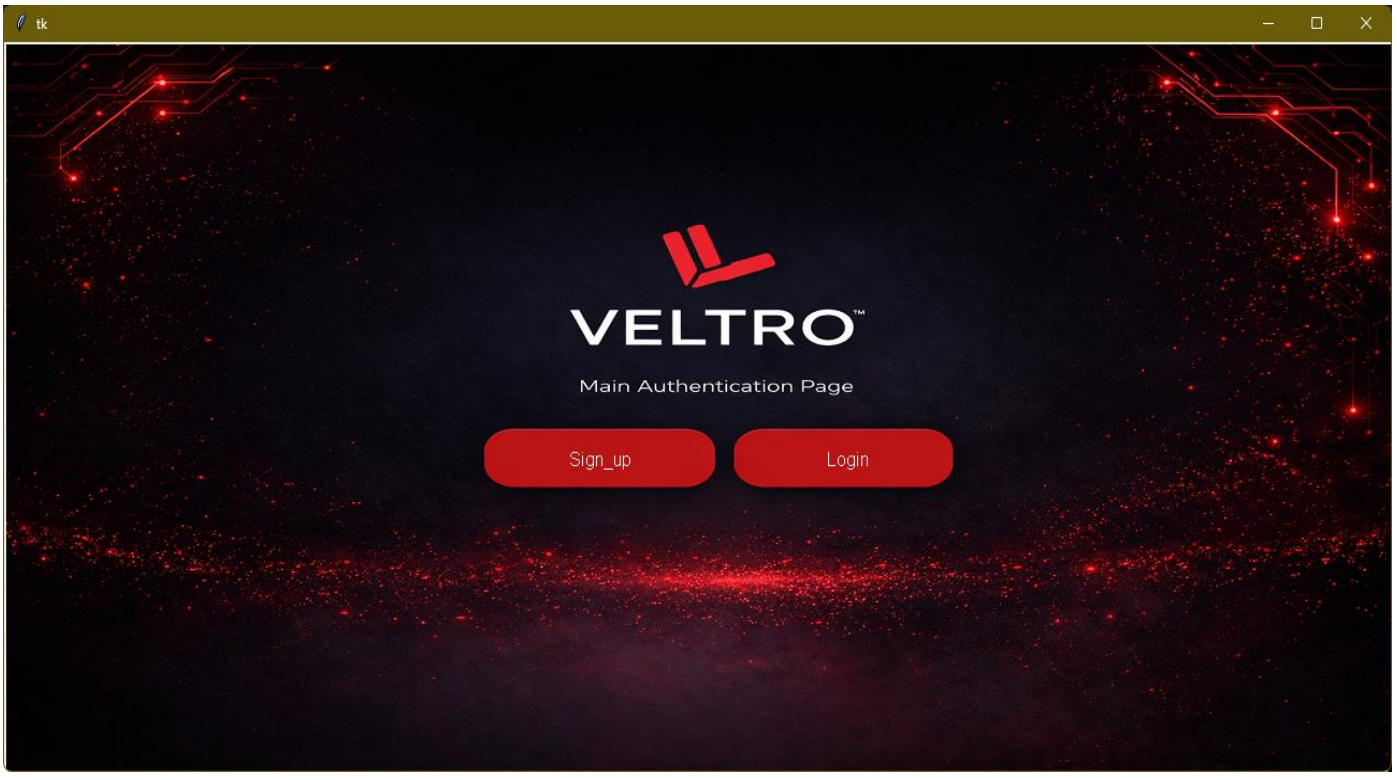
    curr.execute("""
        SELECT category, MIN(Dot), SUM(amount)
        FROM expenses
        WHERE Uid=%s
        GROUP BY category
    """, (username,))

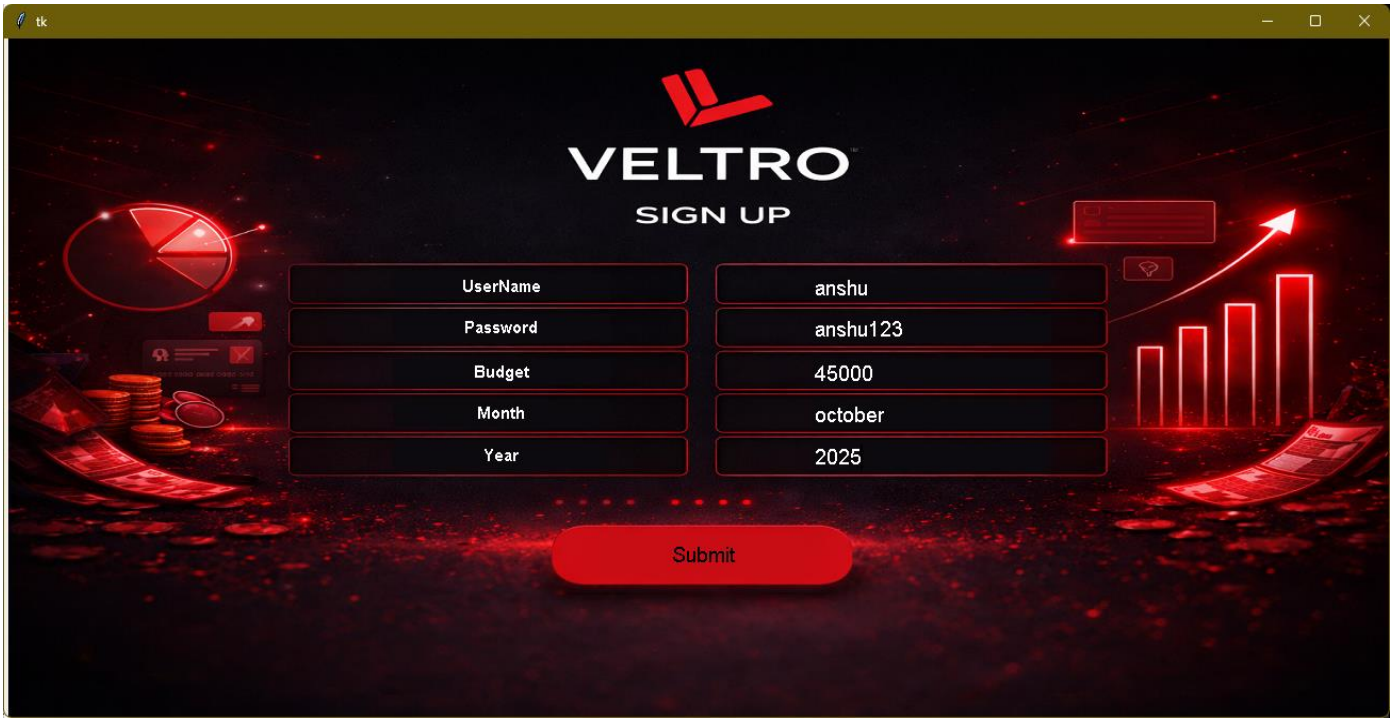
    for row in curr.fetchall():
        tree.insert("", END, values=row)

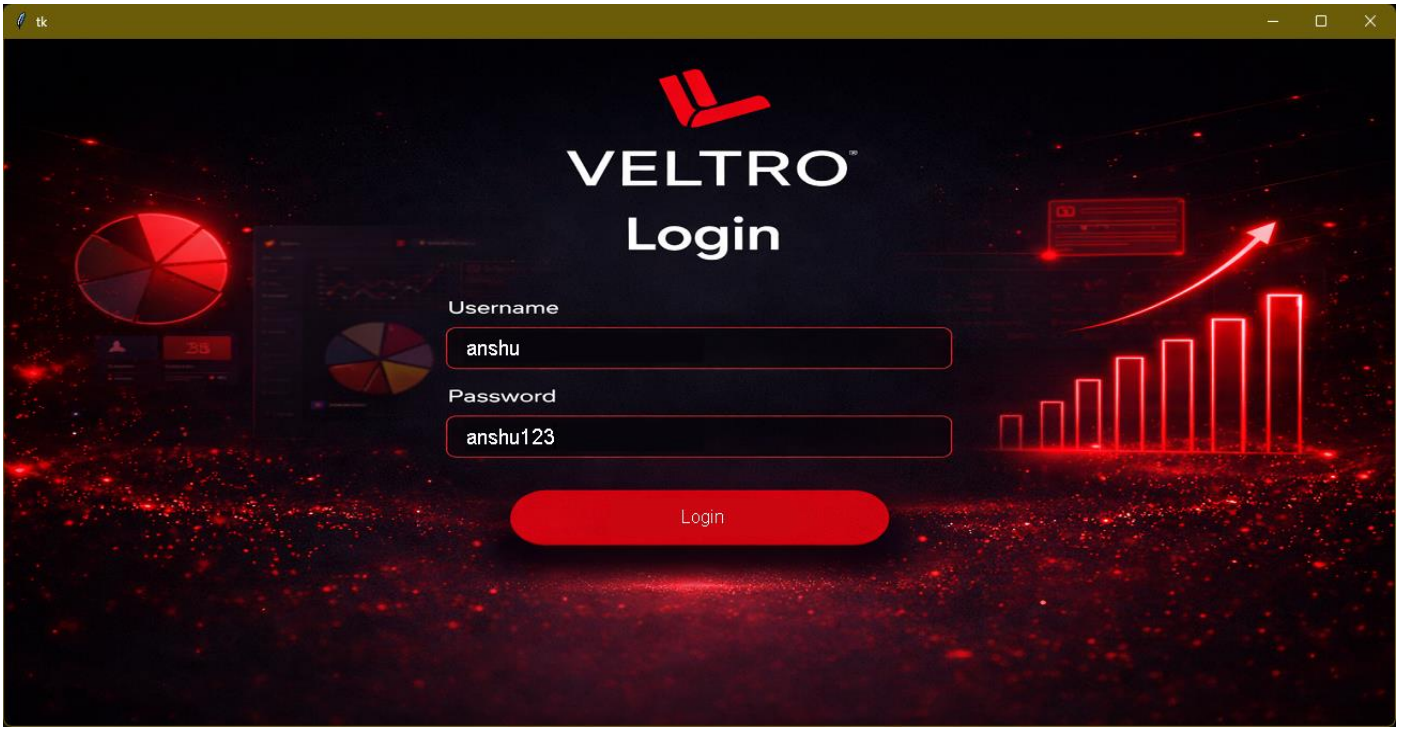
    if totale > budget:
        f19.status_img = PhotoImage(file="budget1.png")
    elif totale == budget:
        f19.status_img = PhotoImage(file="budget2.png")
    else:
        f19.status_img = PhotoImage(file="budget3.png")    Label(f19, image=f19.status_img,
bg="lavender").place(x=480, y=60)
```

OUTPUT


tk
Veltro
Master your expenses with
precision and style
Get Started


tk
VELTRO™
Main Authentication Page
Sign_up
Login


tk
VELTRO
SIGN UP
UserName
anshu
Password
anshu123
Budget
45000
Month
october
Year
2025
Submit

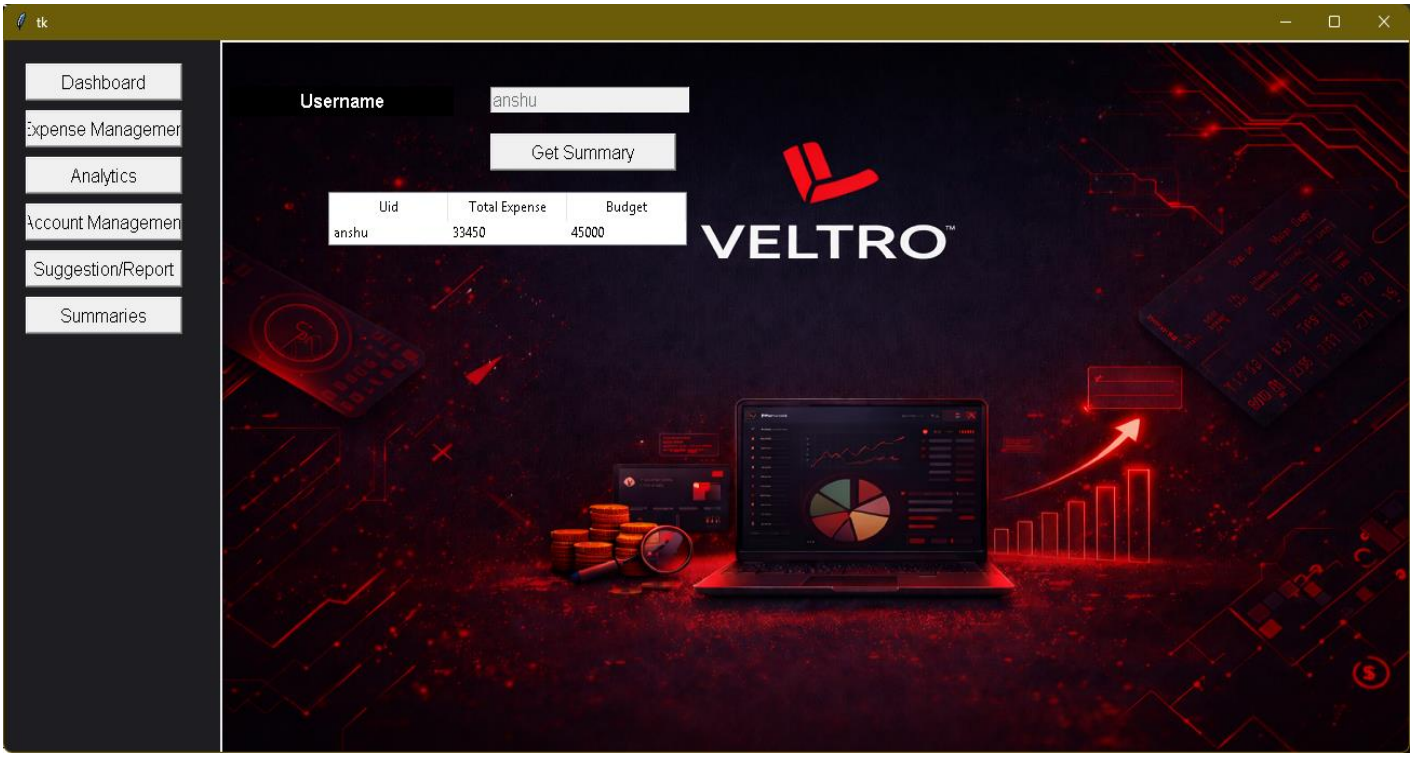

tk
VELTRO
Login
Username
anshu
Password
anshu123
Login

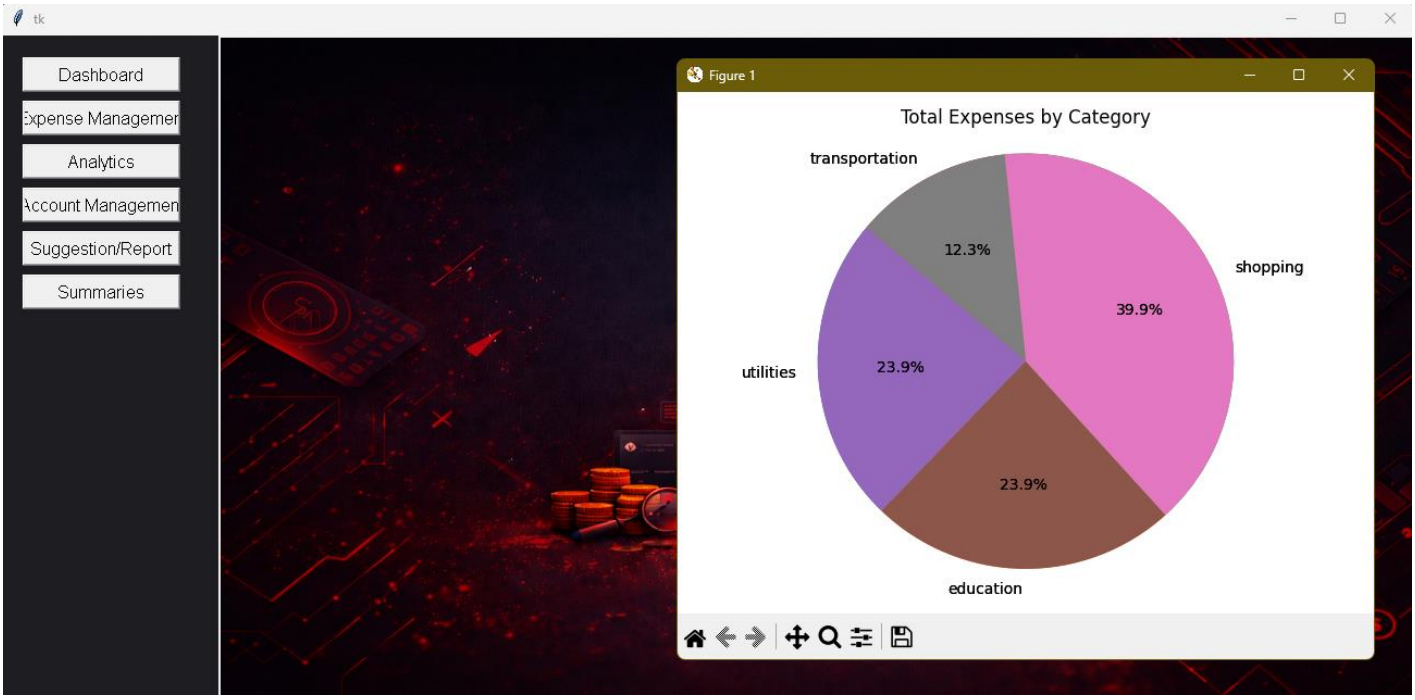
➢ Main page



➢ Dashboard

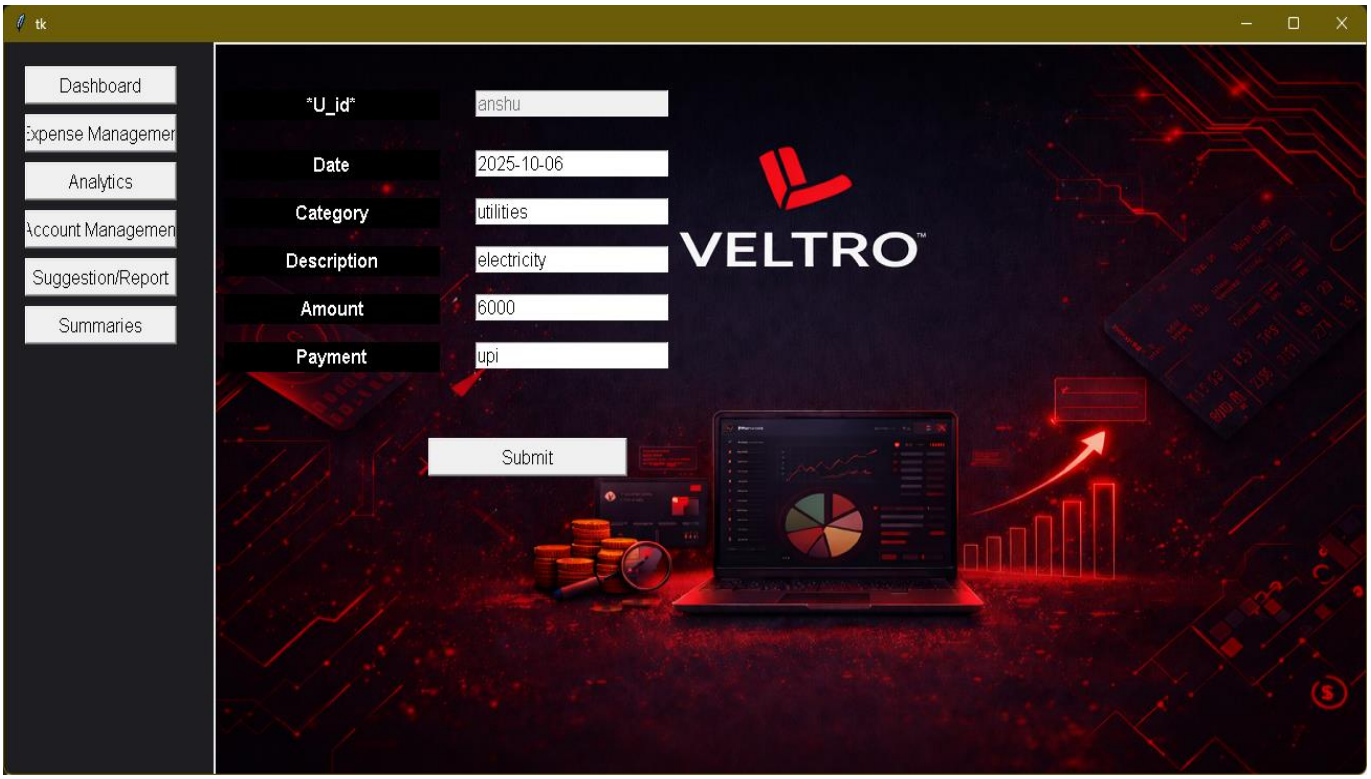


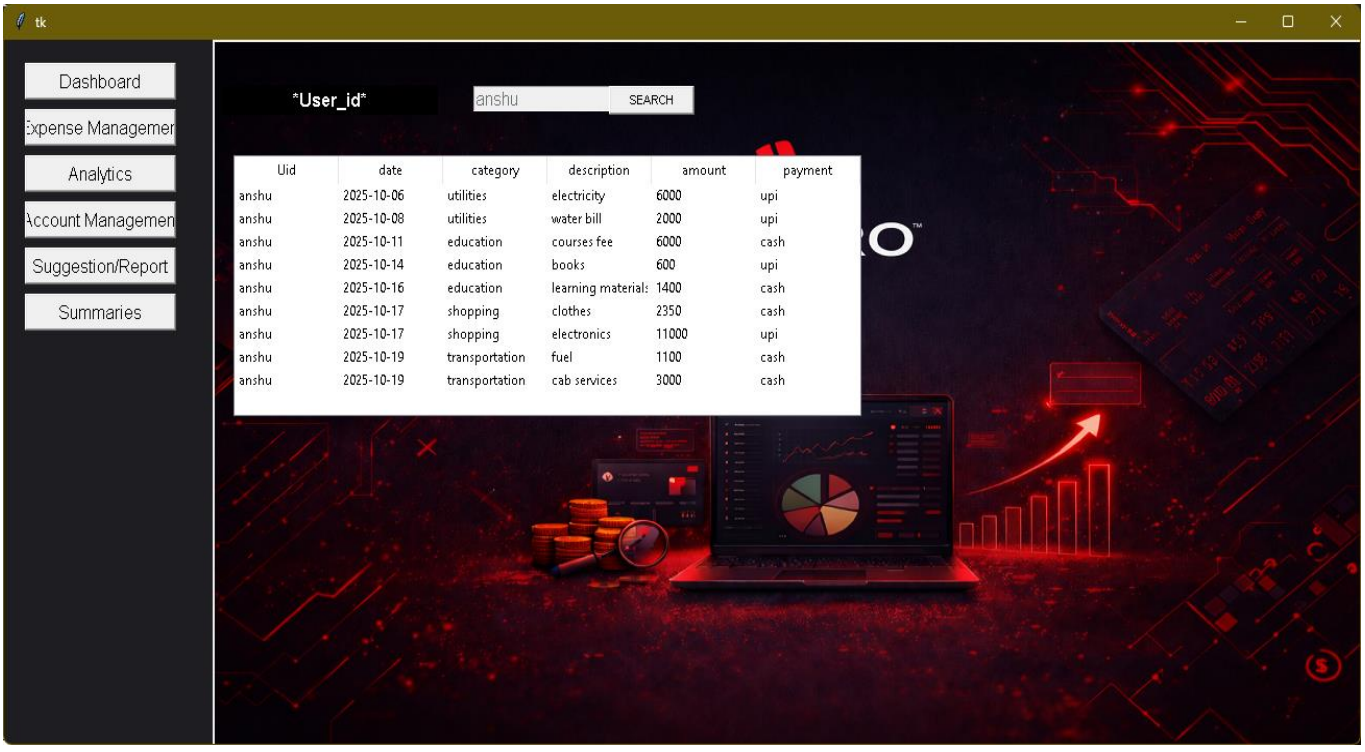

tk
Dashboard
Expense Managemer
Analytics
Account Managemen
Suggestion/Report
Summaries
Username
anshu
Get Summary
Uid
Total Expense
Budget
anshu
33450
45000
VELTRO™


tk
Dashboard
Expense Managemer
Analytics
Account Managemen
Suggestion/Report
Summaries
Figure 1
Total Expenses by Category
transportation
12.3%
shopping
39.9%
utilities
23.9%
23.9%
education

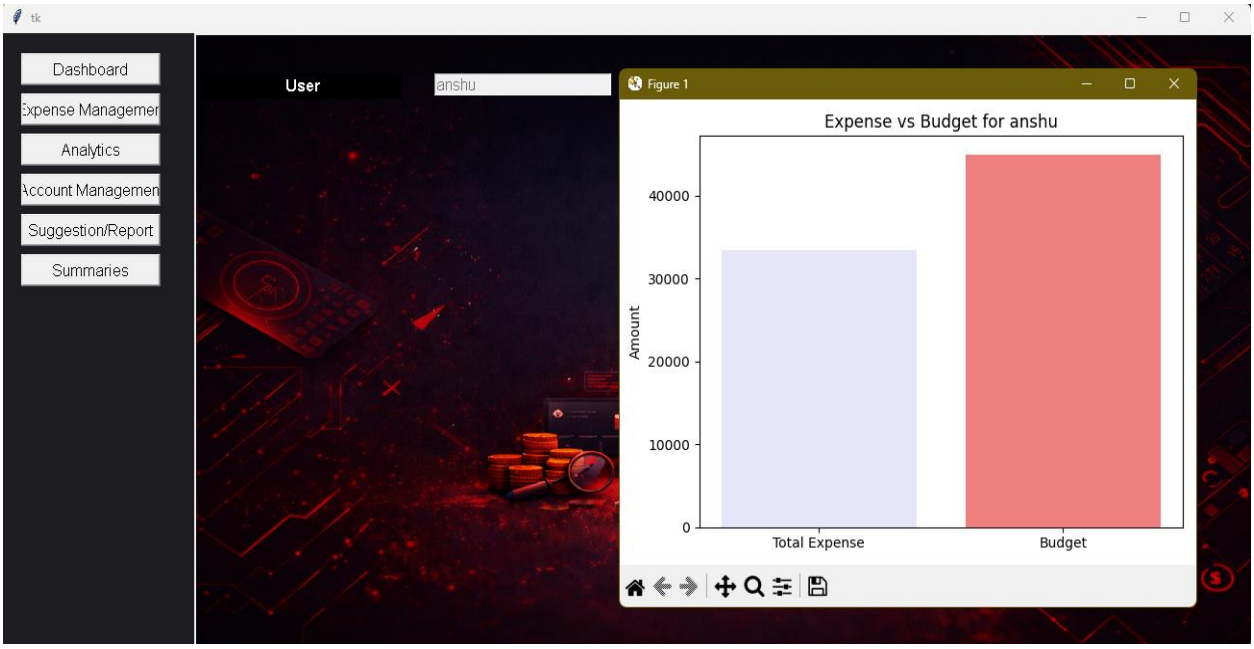
➢ Expense management




tk
Dashboard
Expense Managemer
Analytics
Account Managemen
Suggestion/Report
Summaries
U_id
anshu
Date
2025-10-06
Category
utilities
Description
electricity
Amount
6000
Payment
upi
Submit
VELTRO

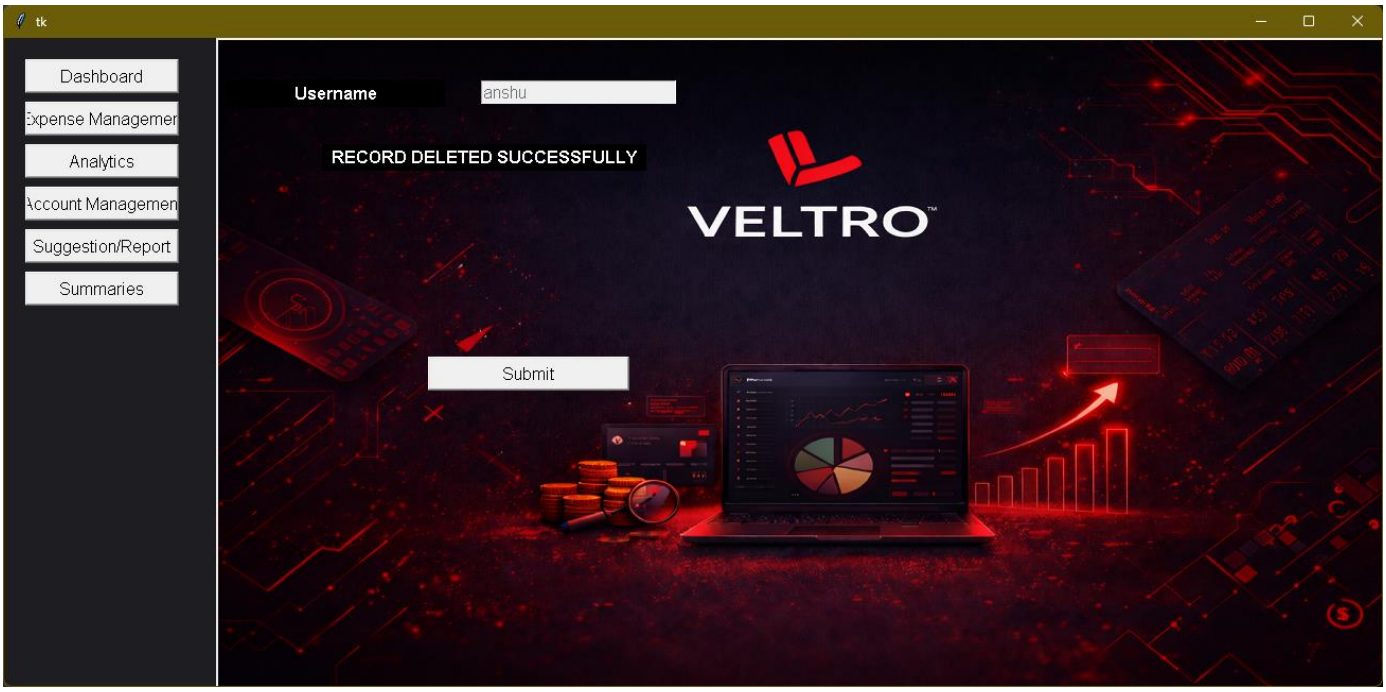

tk
Dashboard
Expense Managemer
Analytics
Account Managemen
Suggestion/Report
Summaries
User_id
anshu
SEARCH
Uid date category description amount payment
anshu 2025-10-06 utilities electricity 6000 upi
anshu 2025-10-08 utilities water bill 2000 upi
anshu 2025-10-11 education courses fee 6000 cash
anshu 2025-10-14 education books 600 upi
anshu 2025-10-16 education learning materials 1400 cash
anshu 2025-10-17 shopping clothes 2350 cash
anshu 2025-10-17 shopping electronics 11000 upi
anshu 2025-10-19 transportation fuel 1100 cash
anshu 2025-10-19 transportation cab services 3000 cash

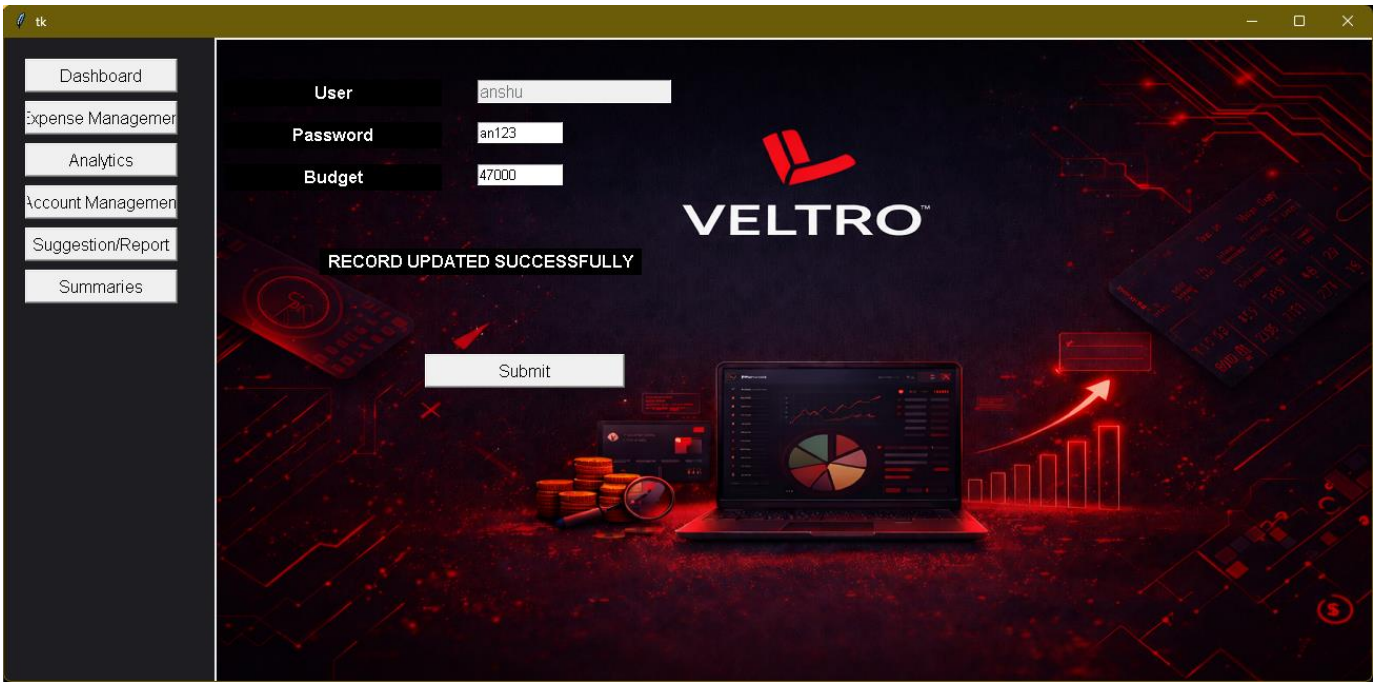
➢ Analytics



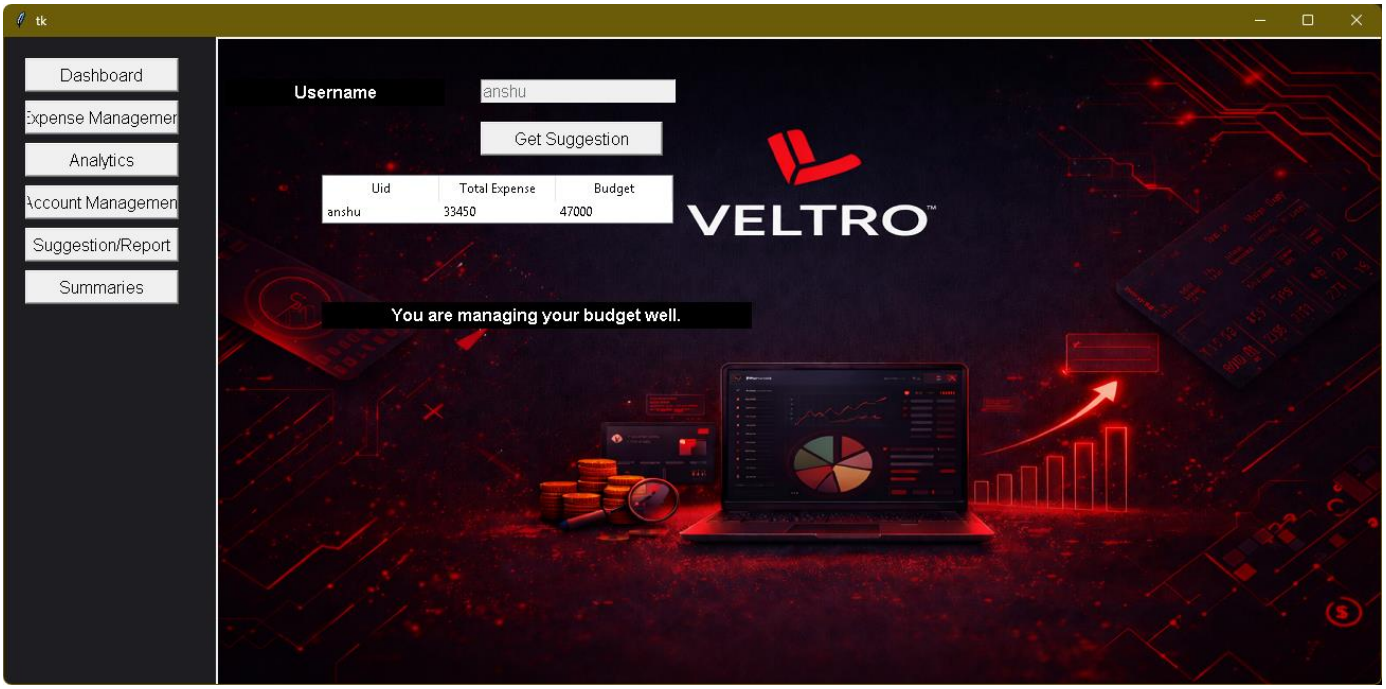
➢ Account management

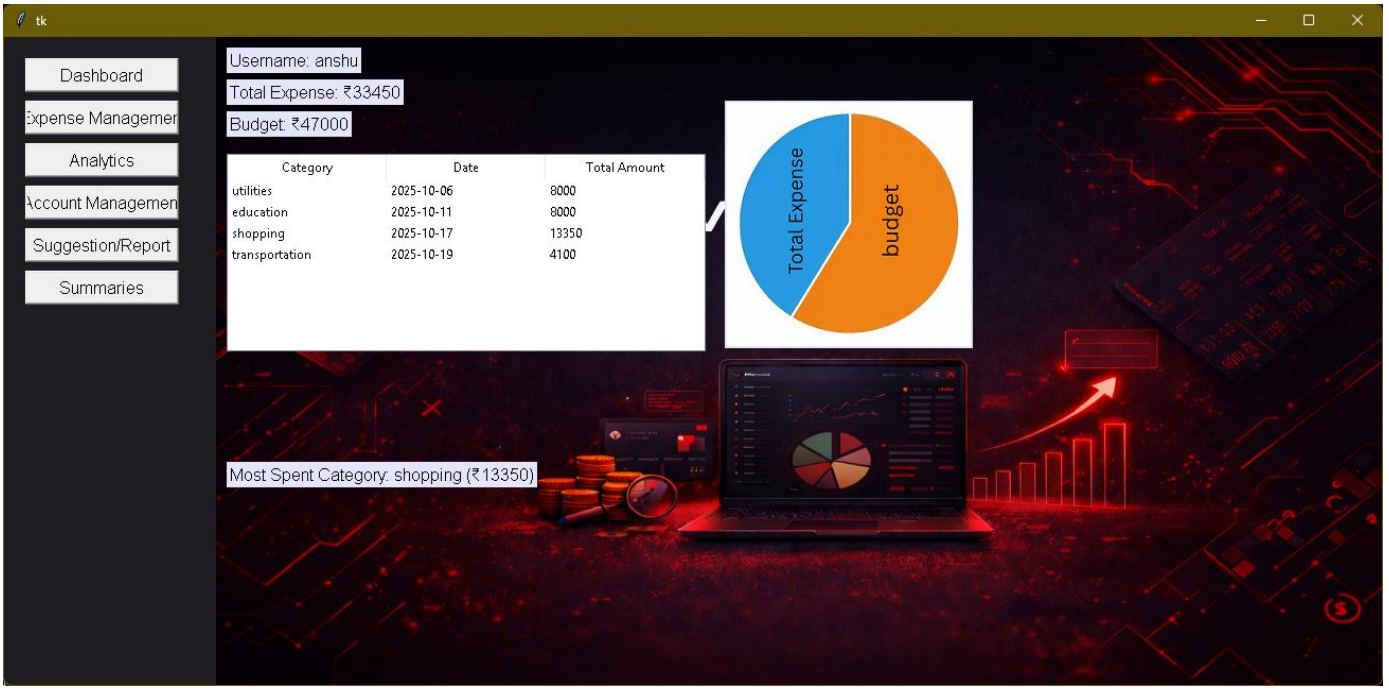



tk
Dashboard
Expense Managemer
Analytics
Account Managemen
Suggestion/Report
Summaries
Username
anshu
RECORD DELETED SUCCESSFULLY
VELTRO
Submit


tk
Dashboard
Expense Managemer
Analytics
Account Managemen
Suggestion/Report
Summaries
User
anshu
Password
an123
Budget
47000
RECORD UPDATED SUCCESSFULLY
VELTRO
Submit

➢ Suggestion and summaries





Bibliography

- **Treofinance – How to Use Expense Tracker Apps Effectively**

https://treofinance.com/how-to-use-expense-tracker-apps-effectively

- **Behance – Expense Tracker App Case**
  https://www.behance.net/search/projects/Expense%20Tracker%20App